

2025 年面向对象程序设计期末考试回忆版（仅编程部分）参考答案

第一题

这一题主要考察模板函数、迭代器解引用、函数对象或 Lambda 的调用，以及“遍历区间并累加”的基本写法。题目虽然把函数名写成了 transform，但它的行为更接近“先对每个元素做变换，再把结果累加起来”。也就是说，它并不是标准库 `std::transform` 的简单照搬，而是把“变换”和“求和”合在了一起。

题意可以概括成下面这几步：

1. 从 `first` 走到 `last`
2. 每次取出当前元素 `*first`
3. 调用一元函数 `func(*first)`
4. 把结果累加到 `init`
5. 最后返回累计值

只要把这几步按模板函数的形式写出来即可。参考实现如下：

```
#include <iostream>
#include <vector>
using namespace std;

template <class In, class T, class Func>
T transform(In first, In last, T init, Func func)
{
    while (first != last) {
        init += func(*first);
        ++first;
    }
    return init;
}

int main()
{
    vector<int> v;
    v.push_back(1);
    v.push_back(2);
    v.push_back(3);
    v.push_back(4);

    int res = transform(v.begin(), v.end(), 0,
        [](int x) { return x * x; });

    cout << res << endl;
    return 0;
}
```

```
}
```

程序运行时，依次计算：

$$1^2 + 2^2 + 3^2 + 4^2 = 1 + 4 + 9 + 16 = 30$$

因此输出：

30

这里最关键的语句是：

```
init += func(*first);
```

它同时体现了三个要点：

1. 通过迭代器 `first` 访问当前元素，要先解引用
2. 对当前元素应用一元函数 `func`
3. 把结果继续累计到 `init`

如果把这题换成别的一元函数，例如“乘 2”“取绝对值”“判断奇偶后返回 0 或 1”，这个模板函数仍然能工作，这正是泛型函数的意义所在。

第二题

这一题主要考察继承、虚函数、多态、基类指针容器以及虚析构函数。题目要求设计一个智能家居系统，最自然的做法就是：

1. 定义一个基类 `SmartDevice`，把设备共有的数据和操作放进去
2. 再让 `SmartThermostat` 和 `SmartLight` 继承这个基类
3. 用虚函数 `info()` 让不同设备输出各自不同的信息
4. 在 `SmartHome` 中保存 `SmartDevice*`，从而统一管理各种设备

因为 `SmartHome` 中保存的是基类指针，所以基类析构函数必须写成虚函数，否则通过基类指针删除派生类对象时会出问题。这一点在复习继承与多态时非常值得特别留意。

参考实现如下：

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;

class SmartDevice {
public:
    SmartDevice(const string& n) : name(n), status(false) {}
    virtual ~SmartDevice() {}

    void turnOn() { status = true; }
    void turnOff() { status = false; }
```

```

void printStatus() const
{
    cout << name << " : " << (status ? "on" : "off") << endl;
}

virtual void info() const
{
    cout << "Device: " << name
        << ", status: " << (status ? "on" : "off") << endl;
}

protected:
    string name;
    bool status;
};

class SmartThermostat : public SmartDevice {
public:
    SmartThermostat(const string& n, int t)
        : SmartDevice(n), temperature(t) {}

    void info() const
    {
        cout << "Thermostat: " << name
            << ", status: " << (status ? "on" : "off")
            << ", temperature: " << temperature << endl;
    }

private:
    int temperature;
};

```

```

class SmartLight : public SmartDevice {
public:
    SmartLight(const string& n, int b)
        : SmartDevice(n), brightness(b) {}

    void info() const
    {
        cout << "Light: " << name
            << ", status: " << (status ? "on" : "off")

```

```

        << " , brightness: " << brightness << endl;
    }

private:
    int brightness;
};

class SmartHome {
public:
    ~SmartHome()
    {
        for (std::size_t i = 0; i != devices.size(); ++i)
            delete devices[i];
    }

    void addDevice(SmartDevice* p)
    {
        devices.push_back(p);
    }

    void showAll() const
    {
        for (std::size_t i = 0; i != devices.size(); ++i)
            devices[i]->info();
    }

private:
    vector<SmartDevice*> devices;
};

int main()
{
    SmartHome home;

    SmartDevice* p1 = new SmartThermostat("BedroomThermostat", 26);
    SmartDevice* p2 = new SmartLight("DeskLight", 80);

    p1->turnOn();
    p2->turnOn();

    home.addDevice(p1);
    home.addDevice(p2);
}

```

```
    home.showAll();  
    return 0;  
}
```

这段程序中，多态效果体现在：

```
devices[i]->info();
```

虽然 `devices` 中存放的是 `SmartDevice*`，但真正运行时，如果该指针实际指向的是 `SmartThermostat`，就会调用 `SmartThermostat::info()`；如果指向的是 `SmartLight`，就会调用 `SmartLight::info()`。这就是通过基类指针触发动态绑定的典型写法。

若按上面的 `main()` 运行，输出形式大致如下：

```
Thermostat: BedroomThermostat, status: on, temperature: 26  
Light: DeskLight, status: on, brightness: 80
```

这份设计同时体现了：

1. **封装**：每个类只管理自己的数据和行为
2. **继承**：派生类复用基类中共有的设备名称和开关状态
3. **多态**：通过基类指针统一操作不同类型的设备

因此它与题目的要求是吻合的。若后续还想扩展更多设备，例如 `SmartDoor`、`SmartCamera`，只要继续从 `SmartDevice` 派生并重写 `info()` 即可，整个系统结构不需要推倒重来。